

Embedded System Architecture – Detailed Answers

1. Recognize the suitable architecture (Von Neumann or Harvard) for an embedded system and justify your choice.

Answer: Harvard Architecture

For most embedded systems, **Harvard architecture** is highly suitable because it uses **separate memory and separate buses for instructions and data**.

Main features:

- **Instruction memory** stores program instructions.
- **Data memory** stores variables and data.
- The processor can **fetch an instruction and access data simultaneously**.
- Separate buses reduce memory-access conflicts.
- It provides better **speed and predictable performance**, which is important in real-time embedded systems.

Example:

Microcontrollers such as many **ARM Cortex-M based systems** use a modified Harvard architecture internally.

Advantages:

1. Faster instruction execution.
2. Instruction and data accesses can occur simultaneously.
3. Better performance for real-time applications.
4. Instruction memory can be protected separately from data memory.

Disadvantage:

- Hardware is more complex because separate instruction and data paths/memories are required.

Comparison:

Feature	Von Neumann	Harvard
Instruction memory	Shared	Separate
Data memory	Shared	Separate
Buses	Common	Separate
Speed	Comparatively lower	Higher
Complexity	Simpler	More complex
Typical use	General-purpose systems	Embedded/DSP systems

Key point: For a performance-oriented embedded system, **Harvard architecture is preferred because instruction and data can be accessed simultaneously**.

2. Explain the difference between CISC and RISC architectures.

CISC stands for **Complex Instruction Set Computer**, while **RISC** stands for **Reduced Instruction Set Computer**.

CISC:

CISC processors contain a **large and complex instruction set**. A single instruction may perform several operations and may take multiple clock cycles.

Example: Intel x86 processors.

RISC:

RISC processors use a **small and simple instruction set**. Instructions are generally designed to execute efficiently, often in a small number of clock cycles.

Example: ARM processors.

Difference:

Feature	CISC	RISC
Instruction set	Large	Small
Instruction complexity	Complex	Simple
Instruction length	Often variable	Usually fixed
Execution	May require multiple cycles	Generally efficient/simple
Memory operations	Can operate directly on memory	Usually load/store architecture
Registers	Relatively fewer traditionally	Generally more registers
Hardware	More complex	Simpler
Compiler	Comparatively simpler	Compiler performs more optimization
Example	x86	ARM

Advantages of RISC:

- Faster and predictable instruction execution.
- Simpler processor hardware.
- Efficient pipelining.
- Lower power consumption, making it suitable for mobile and embedded systems.

Advantages of CISC:

- Complex operations can be performed using fewer instructions.
- Good code density.
- Convenient for general-purpose computing.

Key point:

CISC = complex instructions + large instruction set

RISC = simple instructions + smaller instruction set

3. Classify ARM Cortex and Intel x86 processors for mobile and desktop systems.

ARM Cortex and Intel x86 are processor families commonly associated with different computing environments.

ARM Cortex

ARM Cortex processors are based on the **ARM architecture** and are widely used where **low power consumption and efficiency** are important.

Mobile:

Examples:

- ARM Cortex-A series
- Cortex-A53
- Cortex-A55

- Cortex-A78

Application: Smartphones, tablets, smart devices.

Embedded:

Examples:

- Cortex-M series
- Cortex-M0/M3/M4/M7

Application: Microcontrollers, IoT devices, industrial controllers.

Intel x86

Intel x86 processors are traditionally used extensively in **desktop and laptop systems**, where high general-purpose computing performance is required.

Examples:

- Intel Core i3
- Intel Core i5
- Intel Core i7
- Intel Core i9

Application: Desktops, laptops and workstations.

Classification:

ProcessorArchitecture	Common application	
ARM Cortex-A	ARM	Mobile/tablets
ARM Cortex-M	ARM	Embedded systems
Intel Core	x86/x86-64	Desktop/laptop
Intel Xeon	x86/x86-64	Servers/workstations

Main difference:

- **ARM Cortex:** emphasizes **performance per watt and low power**.
- **Intel x86:** traditionally emphasizes **high general-purpose performance and compatibility**.

Important: Modern processor markets overlap more than this traditional classification suggests, but for an exam, **ARM** → **mobile/embedded** and **x86** → **desktop/server** is the standard classification.

4. Paraphrase ARM assembly instructions to add and subtract two registers.

Assume two registers:

- R1 = first value
- R2 = second value
- R0 = result

Addition:

ADD R0, R1, R2

Meaning:

R0 = R1 + R2

Example:

If:

R1 = 10

R2 = 5

After:

ADD R0, R1, R2

R0 = 15

Subtraction:

SUB R0, R1, R2

Meaning:

R0 = R1 – R2

Example:

R1 = 10

R2 = 5

After:

SUB R0, R1, R2

R0 = 5

Important keywords:

- ADD → addition
- SUB → subtraction
- First operand → destination register
- Second and third operands → source registers

5. List the major components of a processor data path.

A **processor datapath** is the collection of hardware components that **store, transfer, and process data during instruction execution**.

Major components:

1. Program Counter (PC)

- Stores the address of the next instruction.

2. Instruction Register (IR)

- Holds the currently fetched instruction.

3. Register File

- Contains general-purpose registers used to store operands and results.

4. ALU (Arithmetic Logic Unit)

- Performs arithmetic and logical operations.

- Examples: addition, subtraction, AND, OR.

5. Multiplexers (MUX)

- Select one input from multiple inputs and send it to the required datapath component.

6. Memory Interface

- Allows the processor to communicate with instruction and data memory.

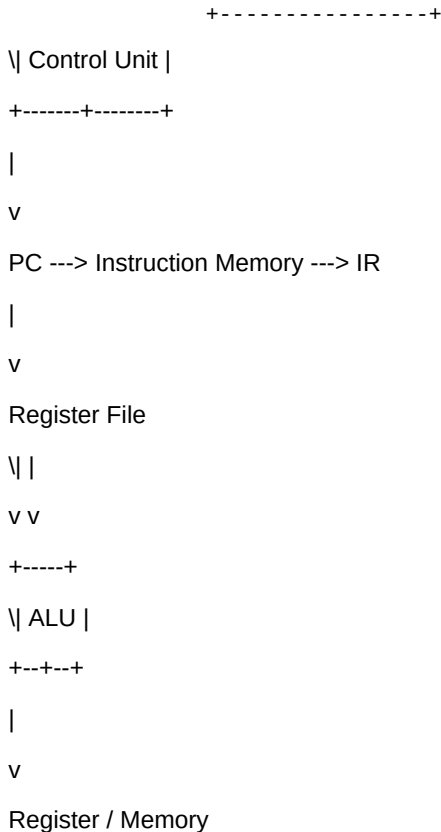
7. Buses / Data paths

- Transfer data between registers, ALU, memory and other components.

8. Control Unit

- Generates control signals that control datapath operations.

Simple representation:



Key point: The datapath performs the actual **movement and processing of data**, while the control unit tells it **what operation to perform**.

6. Identify a simple Verilog statement for AND and OR operations.

In Verilog, **bitwise AND** is represented by `&`, and **bitwise OR** is represented by `|`.

AND:

```
assign y = a & b;
```

This means:

y = a AND b

OR:

```
assign y = a | b;
```

This means:

y = a OR b

Example:

If:

```
a = 1
```

```
b = 0
```

AND:

```
a & b = 0
```

OR:

$$a \mid b = 1$$

Truth table:

ABANDOR			
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	1

Keyword: assign is a continuous assignment statement in Verilog.

7. List the sign, exponent, and mantissa fields in the IEEE-754 format.

The **IEEE-754 standard** defines the representation of floating-point numbers.

For **single precision (32-bit)**:



1. Sign bit — 1 bit

- Indicates whether the number is positive or negative.
- 0 → positive
- 1 → negative

2. Exponent — 8 bits

- Represents the exponent using a **biased representation**.
- Single precision uses a **bias of 127**.

3. Fraction/Mantissa — 23 bits

- Stores the fractional/significand portion of the number.
- For normalized numbers, there is an **implicit leading 1**.

Total:

$$1 + 8 + 23 = 32 \text{ bits}$$

Example:

For a normalized floating-point number:

$$(-1)^{\text{Sign}} \times 1.\text{Fraction} \times 2^{(\text{Exponent} - 127)}$$

Key memory trick:

IEEE single precision = 1 sign + 8 exponent + 23 fraction = 32 bits.

8. State the purpose of the microinstruction and the instruction execution stage.

Microinstruction

A **microinstruction** is a control word used by a microprogrammed control unit to specify the **micro-operations/control signals** that must be performed during a processor cycle.

Purpose:

- Generates control signals.

- Controls registers, ALU, buses and memory.
- Specifies small internal operations required to execute a machine instruction.

Example:

Register → ALU → Register

A microinstruction can control the transfer and ALU operation involved in this process.

Instruction Execution Stage

The **execution stage** is the stage in which the processor **performs the operation specified by the instruction**.

Examples:

- ADD → ALU performs addition.
- SUB → ALU performs subtraction.
- LOAD → data is transferred from memory to a register.
- STORE → data is transferred from a register to memory.

Typical instruction cycle:

Fetch → Decode → Execute → Memory → Write Back

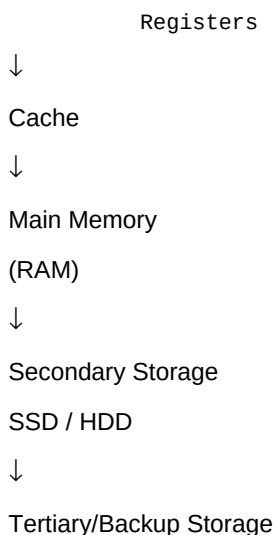
Key point:

Microinstruction controls internal processor operations, while the execution stage actually performs the instruction's required operation.

9. Recall the levels of the memory hierarchy.

The **memory hierarchy** organizes memory according to **speed, cost, capacity and proximity to the CPU**.

From fastest/smallest to slowest/largest:



1. Registers

- Fastest memory.
- Located inside the CPU.
- Very small capacity.
- Stores operands and intermediate results.

2. Cache

- Very fast memory between CPU and main memory.

- Usually divided into L1, L2 and sometimes L3.
- Stores frequently accessed data/instructions.

3. Main Memory

- Usually RAM.
- Larger than cache but slower.
- Holds currently running programs and data.

4. Secondary Storage

- SSD/HDD.
- Non-volatile.
- Large capacity.
- Slower than RAM.

Principle:

As we move down the hierarchy:

- Speed ↓
- Cost per bit ↓
- Capacity ↑
- Access time ↑

10 & 11. Classify the difference between SRAM, DRAM and Flash memory.

These are different types of memory with different storage mechanisms and applications.

Feature	SRAM	DRAM	Flash
Full form	Static RAM	Dynamic RAM	Flash memory
Volatile?	Yes	Yes	No
Storage	Flip-flop	Capacitor + transistor	Floating-gate/charge-trap cells
Speed	Very high	High but slower than SRAM	Slower than RAM
Refresh required	No	Yes	No
Density	Lower	Higher	Very high
Cost/bit	High	Lower	Low
Main application	Cache	Main memory	SSD, USB, memory cards

SRAM

Static Random Access Memory stores each bit using a flip-flop circuit.

- Does not require periodic refresh.
- Very fast.
- Expensive and consumes more chip area per bit.
- Used mainly for **CPU cache memory**.

DRAM

Dynamic Random Access Memory stores each bit using a capacitor and transistor.

- Requires periodic **refreshing**.

- Slower than SRAM.
- Higher density and cheaper per bit.
- Used as **main system memory/RAM**.

Flash Memory

Flash is a **non-volatile semiconductor memory**.

- Retains data even when power is removed.
- Used in **SSDs, USB drives, memory cards and embedded storage**.
- Has limited write/erase endurance compared with RAM.

Easy memory trick:

SRAM → Cache

DRAM → Main RAM

Flash → Storage

12. Define Instruction Set Architecture (ISA). Mention any two design principles of ISA.

Definition:

Instruction Set Architecture (ISA) is the interface between **software and hardware** that defines the instructions, registers, data types, addressing modes and memory-access mechanisms supported by a processor.

In simple words:

*ISA tells the programmer/compiler **what the processor can do and how instructions communicate with the hardware.***

Examples:

- ARM ISA
- x86 ISA
- RISC-V ISA

Design principles of ISA:

Any two can be written:

1. Simplicity

Instructions should be simple and easy to implement efficiently.

2. Regularity

Instructions should follow consistent formats and conventions, making them easier for hardware and compilers to use.

Other valid principles:

- Provide efficient instruction encoding.
- Support efficient addressing modes.
- Provide sufficient registers.
- Maintain compatibility and extensibility.
- Support efficient compiler implementation.

Exam keyword:

ISA = instruction set + registers + data types + addressing modes + programmer-visible processor behavior.

13. What are addressing modes? Name any two addressing modes with examples.

Definition:

Addressing modes specify how an instruction identifies or obtains its operand/data.

They tell the processor where the required data is located.

1. Immediate Addressing

The operand value is directly included in the instruction.

Example:

```
MOV R1, #10
```

Here, 10 is the actual value.

Advantage: Fast because no separate memory access is required for the operand.

2. Register Addressing

The operand is stored in a processor register.

Example:

```
ADD R0, R1, R2
```

Here, R1 and R2 contain the operands.

Advantage: Very fast because registers are inside the CPU.

Other addressing modes:

- Direct addressing
- Indirect addressing
- Register indirect
- Indexed addressing
- Base + offset addressing
- Relative addressing

Key point: Addressing modes determine where the operand comes from.

14. A processor executes 2×10^9 instructions with a CPI of 2 and clock rate of 2 GHz. Calculate the execution time.

Given:

Number of instructions:

$I = 2 \times 10^9$

CPI:

$CPI = 2$

Clock rate:

$2\text{GHz} = 2 \times 10^9\text{Hz}$

Formula:

$\text{CPU Time} = \text{Clock Rate} \times \text{Instruction Count} \times \text{CPI}$

Substitute:

CPU Time= $2 \times 10^9 (2 \times 10^9)^2 = 2 \times 10^9 \times 10^9 = 2$ seconds

Answer:

2 seconds

Extra verification:

Total clock cycles:

$2 \times 10^9 \times 2 = 4 \times 10^9$ cycles

At a clock rate of 2×10^9 cycles/second:

$4 \times 10^9 \div 2 \times 10^9 = 2$ s

Final answer: Execution time = 2 seconds.

15. What is the purpose of an FSM in processor control?

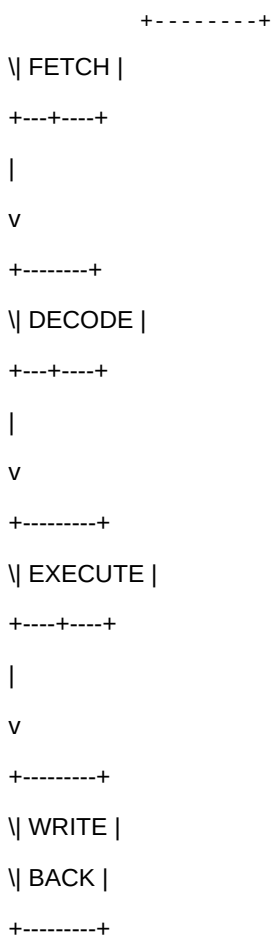
FSM stands for **Finite State Machine**.

An FSM is used in processor control to represent the **different states of instruction execution and the transitions between those states**.

Purpose:

- Controls the sequence of processor operations.
- Generates appropriate control signals.
- Coordinates stages such as fetch, decode, execute and memory access.
- Determines the next state based on the current state and inputs.

Example:



Applications:

FSM-based control is commonly used in:

- Processor control units
- Instruction sequencing
- Bus controllers
- Communication controllers

Advantages:

- Easy to understand and design.
- Provides predictable control sequencing.
- Useful for implementing synchronous control logic.

Key point:

FSM controls the sequence of operations by moving through predefined states.

16. Locate a simple Verilog statement for AND and OR operations.

This is essentially the same concept as Question 6.

AND operation:

```
assign y = a & b;
```

OR operation:

```
assign y = a | b;
```

Example:

```
module logic_operations(  
input a,  
input b,  
output y_and,  
output y_or  
);  
assign y_and = a & b;  
assign y_or = a | b;  
endmodule
```

Truth table:

ABA AND BA OR B			
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	1

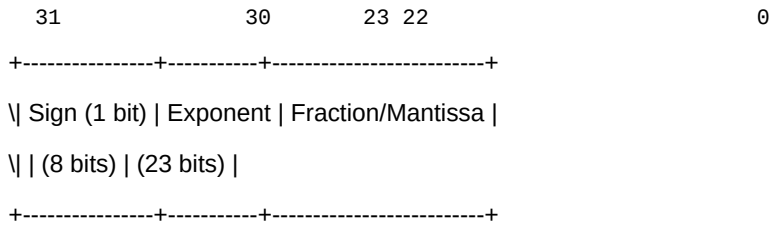
Remember:

& → AND

| → OR

17. State the IEEE 754 single-precision floating-point format and its three fields.

The **IEEE-754 single-precision format** represents a floating-point number using **32 bits**.



Three fields:

1. Sign — 1 bit

- 0 → positive
- 1 → negative

2. Exponent — 8 bits

- Uses a **bias of 127**.
- Actual exponent:

$$E = \text{Exponent} - 127$$

for normalized values.

3. Fraction/Mantissa — 23 bits

- Stores the fractional part of the significand.
- Normalized numbers have an implicit leading 1.

Formula:

$$(-1)^S \times 1.F \times 2^{(E-127)}$$

Total:

$$1 + 8 + 23 = 32 \text{ bits}$$

Exam answer keyword:

Single precision = Sign(1) + Exponent(8) + Fraction(23).

18. State the purpose of the microinstruction and the instruction execution stage.

Microinstruction

A **microinstruction** is a binary/control word that specifies the **micro-operations and control signals** required inside the processor.

It may control:

- Register transfers
- ALU operation
- Memory access
- Bus selection
- Register loading

Instruction Execution Stage

The **execution stage** performs the actual operation specified by the decoded instruction.

Examples:

ADD → ALU performs addition

SUB → ALU performs subtraction

LOAD → memory data goes to register

STORE → register data goes to memory

Instruction cycle:

Fetch → Decode → Execute → Memory → Write Back

In one line:

Microinstruction generates internal control actions; the execution stage performs the instruction's required operation.

19. Discuss the levels of the memory hierarchy.

Memory hierarchy is an arrangement of storage levels based mainly on **speed, capacity and cost per bit**.

1. Registers

- Located inside the CPU.
- Fastest storage available to the processor.
- Very small capacity.
- Hold immediate operands and intermediate results.

Application: CPU calculations.

2. Cache Memory

Cache is a high-speed memory located close to or inside the processor.

Types:

- **L1 cache** → smallest and fastest.
- **L2 cache** → larger and slower than L1.
- **L3 cache** → larger and generally slower than L2.

Application: Stores frequently accessed instructions and data.

3. Main Memory

Usually implemented using **DRAM**.

- Larger than cache.
- Slower than cache.
- Volatile.
- Stores currently executing programs and their data.

Application: System RAM.

4. Secondary Storage

Examples:

- SSD

- HDD

Characteristics:

- Non-volatile.
- Large capacity.
- Cheaper per bit.
- Slower than main memory.

Application: Permanent storage of operating systems, applications and files.

5. Tertiary/Backup Storage

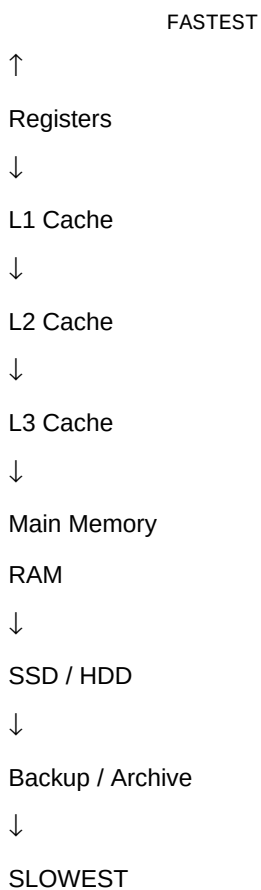
Examples:

- Magnetic tape
- Large-scale archival storage

Used mainly for:

- Backup
- Archiving
- Long-term storage

Overall hierarchy:



As we go downward:

Capacity increases ↑

Cost per bit decreases ↓

Speed decreases ↓

Access time increases ↑

Why hierarchy is needed?

A system cannot practically build all memory using the fastest technology because it would be **too expensive and limited in capacity**.

Therefore, computers combine:

- small, fast, expensive memory
- large, slower, cheaper memory

to obtain a good balance between **performance, cost and capacity**.

20. Identify SRAM and DRAM with respect to speed and application.

Both SRAM and DRAM are **volatile semiconductor memories**, but their speed and applications are different.

SRAM — Static RAM

Speed:

Very fast

Why?

SRAM stores data using flip-flop circuits and **does not require periodic refresh**.

Application:

Primarily used for **CPU cache memory**.

Examples:

- L1 cache
- L2 cache
- Parts of L3 cache

Advantages:

- Very fast.
- No refresh operation.
- Good for high-speed processor access.

Disadvantages:

- Expensive.
- Requires more transistors per bit.
- Lower storage density.

DRAM — Dynamic RAM

Speed:

Slower than SRAM, but still fast enough for main memory.

Why?

DRAM stores each bit as electrical charge in a capacitor, so the stored charge must be **periodically refreshed**.

Application:

Used as **main memory/system RAM**.

Examples:

- DDR4
- DDR5

Advantages:

- High density.
- Cheaper per bit.
- Suitable for large-capacity memory.

Disadvantages:

- Requires periodic refresh.
- Slower than SRAM.
- Refresh consumes time and power.

SRAM vs DRAM:

Feature	SRAM	DRAM
Speed	Faster	Slower
Refresh	Not required	Required
Storage cell	Flip-flop	Capacitor + transistor
Density	Lower	Higher
Cost/bit	Higher	Lower
Main application	Cache	Main memory
Volatile	Yes	Yes

Easy exam memory:

SRAM = Speed + Cache

DRAM = Density + Main RAM

■ Ultra-quick revision sheet

If you're literally sitting in the exam hall with 3 minutes left, memorize this:

Q	One-line answer
1	Harvard → separate instruction/data memory → simultaneous access → high embedded performance
2	CISC → complex/large instruction set; RISC → simple/reduced instruction set
3	ARM → mobile/embedded; x86 → desktop/server traditionally
4	ADD R0, R1, R2 → R0=R1+R2; SUB R0, R1, R2 → R0=R1-R2
5	PC, IR, registers, ALU, MUX, buses, memory interface, control unit
6	assign y=a&b; and \assign y=a
7	IEEE single: Sign 1 + Exponent 8 + Fraction 23
8	Microinstruction → generates control signals; Execute → performs instruction operation
9	Registers → Cache → RAM → Secondary storage → Archive
10/11	SRAM → cache; DRAM → main memory; Flash → non-volatile storage

QOne-line answer	
12	ISA = interface between software and hardware
13	Addressing mode = method of specifying operand location
14	CPU time = IC × CPI / Clock rate = 2 seconds
15	FSM → controls sequential states and generates processor control signals
16	& → AND, \`
17	32-bit IEEE single = 1+8+23
18	Microinstruction controls micro-operations; execution stage performs instruction
19	Hierarchy balances speed, capacity and cost
20	SRAM = faster/cache; DRAM = slower/main RAM

Most important keywords to NOT forget in the exam:

Harvard → separate memory/bus

RISC → simple instructions

CISC → complex instructions

ISA → hardware/software interface

FSM → states + transitions + control

SRAM → cache

DRAM → main memory

Flash → non-volatile

IEEE-754 → 1/8/23

CPU time → IC × CPI / clock rate